Reflection Journal

ITAI 1371 Machine Learning

Midterm Project: Stroke Risk Prediction

Dataset	healthcare_disease_prediction_2000.csv (2,000 rows); target: Stroke
GitHub repository	ITAI-1371-ML / Midterm Project
Submission date	10/28/2026
Team	Camila, Danh, Great, Karima, Quan

Every team member writes their own reflection in their own words.

Write in full sentences and connect what you learned to what you actually did in the project.

Karima — Data & Split Lead

This project was a valuable learning experience for me because it helped me understand more about the practical steps involved in machine learning. Before working on this assignment, I did not fully understand how important data preparation is before building a machine learning model. I learned that we cannot simply take a dataset and start training a model without checking whether the data is organized correctly, whether there are missing values, and how the data should be divided. Each step has a purpose and can affect the quality and reliability of the final results.

One of the most important things I learned was how to divide a dataset into 70% training data and 30% testing data using Python's train_test_split. I understand that the training data is used to help a model learn patterns, while the testing data is used to evaluate how well the model performs on data it has not seen before. At first, I found this process confusing because I was not completely sure how the split worked or why we needed two separate datasets. However, as I learned more about the process, I began to understand why keeping the training and testing data separate is important.

I also learned about missing values and how to inspect them in a dataset. Even though our dataset did not contain missing values according to the report, checking for them was still an important part of the assignment. I learned that if missing values are present, we may need to use an imputation method to fill them in. One concept I found particularly difficult was understanding why the imputer must be fitted on the training data only and then used to transform the testing data. I learned that this helps prevent data leakage, which can make a machine learning model's performance appear better than it really is.

Another important lesson was understanding exploratory data analysis, or EDA. I learned that descriptive statistics, such as .describe(), and frequency counts using value_counts() can help us understand the information in a dataset. However, I also learned that, for our assignment, these analyses should be performed only on the training data. Understanding why we need to follow this rule took some extra effort, but it helped me see how important it is to follow the correct process when preparing data for machine learning.

Personally, I found some parts of this project challenging, especially understanding the Python code and connecting each step to the overall machine learning process. Sometimes, I needed more time to understand what the code was doing and why a particular step was necessary. This made me realize that learning machine learning requires patience, practice, and a willingness to ask questions when something is unclear. Even when I found a concept difficult, I tried to focus on understanding its purpose instead of simply memorizing the steps.

Overall, this project helped me identify both my strengths and the areas where I need improvement. I still need more practice with Python, data analysis, and understanding technical concepts independently, but I feel that I have gained a better foundation than I had before starting the assignment. In future projects, I want to become more confident writing and understanding code, interpreting statistical results, and explaining why each step is necessary. I also want to improve my problem-solving skills so that I can work more independently instead of feeling confused when I encounter an unfamiliar task.

Most importantly, I learned that making mistakes or finding something difficult does not mean that I cannot learn it. It means I need more practice and time to understand it properly. This project reminded me that learning is a process, and every assignment gives me an opportunity to improve. I am glad I had the chance to work on this project because it helped me understand the importance of proper data preparation and gave me more confidence to continue learning about machine learning.

Danh — EDA Lead

My part was the EDA, and I only worked with the 1,400-row training set from Karima's split. I added a quick check in my code so it would stop if someone accidentally loaded the full 2,000 rows. I started with the basics, like .describe() and value counts, and right away noticed that only about 5% of patients had a stroke, which is only 71 people. That told me balancing was going to matter a lot.

After that, I made the charts. I used Seaborn for the box and violin plots of the vitals and the correlation heatmap, and I also made an age histogram split by stroke and a class balance bar chart with plain Matplotlib. I ran an outlier check, and some of the data looked off. There were 10 BMI values that were way too low, and 124 rows where diastolic was equal to or higher than systolic, which doesn't really happen in real life. I also checked skewness and found that the vitals were close to normal, so a log transform wasn't needed. I sent these notes to Great for scaling.

The part that surprised me most was finding out that Stroke follows a simple rule: over 60, systolic over 150, and a smoker. Every stroke case fits that rule, so the dataset is clearly made up, and we noted it as a limitation. I also caught a small typo in the imports and some lost formatting when we put everyone's code together.

Great — Encoding, Scaling & Normalization

Working on this data pipeline provided me with valuable experience in data preparation for machine learning. The pipeline involves extracting clinical metrics from messy, raw data to build features that can then be used for training. I learned that data leakage can be very insidious. Sometimes data leakage is as simple as running a fit or transformation across the whole data set. With practice, I learned how to ensure complete data leakage separation by strictly enforcing boundaries between my X_train and X_test data, and checking that the model parameters are also independent across all pipeline steps. I also learned that in conjunction with pipeline isolation, feature engineering has greater predictive power than inputting raw data to the machine learning algorithm.

Beyond the code, the pipeline taught me about real-world applications of decision-making, analysis, and problem-solving. Avoid data leakage in business by relying only on the data at hand at a particular point in time and eliminating the retrospective, possibly biased data, with real-world decision-making. Similar to the machine learning task, feature engineering condenses data to clear metrics and real world metrics report data to KPIs that are clean and actionable. The final data leakage example teaches us to consider skewed data and avoid real-world metrics based on averages when data (i.e. incomes, risk and behavior) is not typically normal.

Using unit tests with pytest and documenting our methodology transformed my outlook on creating software that functions in the long term, rather than just writing code to meet immediate functional needs. It's important to create safeguards and validate assumptions to ensure that a system will be able to perform under heavy loads, or when team members start working with that system in different contexts. A software system with a high level of process isolation and data interpretation, along with built in quality assurance systems, is essential for every software engineer and systems architect in the modern age.

Quan — Feature Engineering & Balancing

Before this project, I thought machine learning was mostly about picking a model. I learned that most of the work happens before that, in getting the data ready. We started by splitting the data 70% for training and 30% for testing, and from then on we only explored and learned from the training data so the test set stayed untouched. Then we filled in impossible values like very low BMI, turned Gender into numbers, scaled the measurements so they were in the same range, and balanced the classes. Seeing the before-and-after plots for each step made it clear what each one actually did to the data.

My part was feature engineering and class balancing. I created new features from the columns we already had, such as pulse pressure, a blood pressure risk category and a BMI category, so the model could see patterns that a doctor would look for. The hardest part was the imbalance. Only about 5% of patients had a stroke, so a model could guess "no stroke" every time and still look 95% accurate. I used SMOTE-NC to create extra stroke examples, and I learned it must only be used on the training data, because adding fake patients to the test set would make the results meaningless. Choosing SMOTE-NC instead of plain SMOTE also taught me to check that the new data makes sense, since our data mixes numbers and yes/no columns.

What I take away is that clean data and honest evaluation matter more than a fancy model. Our models scored very high, which made us suspect the dataset was generated from simple rules, so good scores don't always mean a model would work on real patients. Next time I would test whether each new feature really helps instead of just adding it, and I would talk with the team earlier about which steps depend on each other. For the final project I feel more ready, because we have a clean, balanced training set and I understand why each step was done.

Camila — Documentation & GitHub Lead

My role in this project was to help organize the documentation and project files for GitHub. I learned that keeping files organized and explaining the steps clearly is important, because it helps the instructor and other people understand what the group did.

While working on the project, I also learned more about the order of the preprocessing steps. We split the data into training and testing sets, checked the training data, and then applied the preprocessing steps. I understand better now why we should not use the test data during EDA or when fitting the preprocessing tools.

One thing that surprised me was that a simple rule matched all the stroke cases in the training data. This helped me understand that very high model scores do not always mean a model will work well with real patient data, since the dataset itself can affect the results.

I still want to improve my confidence when explaining technical ideas, but I understand more now about how documentation supports a machine learning project.